Listing E.52 HTML partial to display an ad

```
<!-- RiffMates/templates/partials/ad_results.html -->
{% for ad in ads %}
  <p>
    {{ad.date}} &mdash;
    {{ad.musician.first_name}} {{ad.musician.last_name}}

    <br/>
    {% if ad.seeking == "M" %}
      Band, "{{ad.band.name}}", is seeking a musician
    {% else %}
      Musician, "{{ad.musician.first_name}} {{ad.musician.last_name}}"
      is seeking a band
    {% endif %}
    <br/>
    <i>{{ad.content}}</i>
  </p>
  <hr>
{% empty %}
  <p> <i>No ads at this time</i> </p>
{% endfor %}

{% if has_more %}
  <div
    hx-get="
    ➥ {% url 'search_ads' %}?page={{next_page}}&search_text={{search_text}}"
    hx-trigger="revealed"
    hx-swap="outerHTML">
    <i> Loading more results ... </i>
  </div>
{% endif %}
```

Loop through the ads given, and display them to the user.

The Django paginator indicates if there is more data.

This tag gets replaced when more results are loaded. The hx-get value fetches data from the next page for the given search terms.

The GET is triggered when this tag is displayed onscreen.

A value of outerHTML means the entire tag gets replaced rather than just the contents within the tag.

A placeholder to display until the results have returned.

The view for performing search is responsible for both displaying the main page and a partial for result data. The search text is split into terms, and a `Q` object is built to perform the search. The search results are paginated, and the partials can be rendered one "page" at a time. The following listing contains the view code.

Listing E.53 An HTMX-capable view for searching ads

```
# RiffMates/content/views.py
import urllib
from time import sleep
...

from django.core.paginator import Paginator
from django.db.models import Q
from django.shortcuts import render

from bands.views import _get_items_per_page, \
    _get_page_num
...
```

Import the utilities for handling pagination from bands/views.py. A better design would move these to a common utilities file.